SCOOT LEARNING LAB · STUDY GUIDE

Three small optimisers that run a whole city's lights

How SCOOT turns its link models into coordinated splits, offsets and cycle times, and copes with jams, buses and broken detectors.

Level 2 · Advanced



INSIDE

- Why most of SCOOT's decisions are 'do nothing'
- How a green wave finds itself, 4 seconds at a time
- Why the cycle grows fast in a peak but shrinks slowly
- What happened when we switched each mechanism off in SUMO

Beginner-friendly companion to smrahmanbd.github.io/scoot-learning-lab

The map: SCOOT Advanced

This guide continues from the Foundation guide. There you built SCOOT's model of one link: LPUs, profiles, the queue and the degree of saturation (DoS). Here you will see how SCOOT uses that model to make decisions for a whole group of junctions. You still need only school maths. There are eight ideas; each one is a small, simple rule, and the power comes from running all of them together.

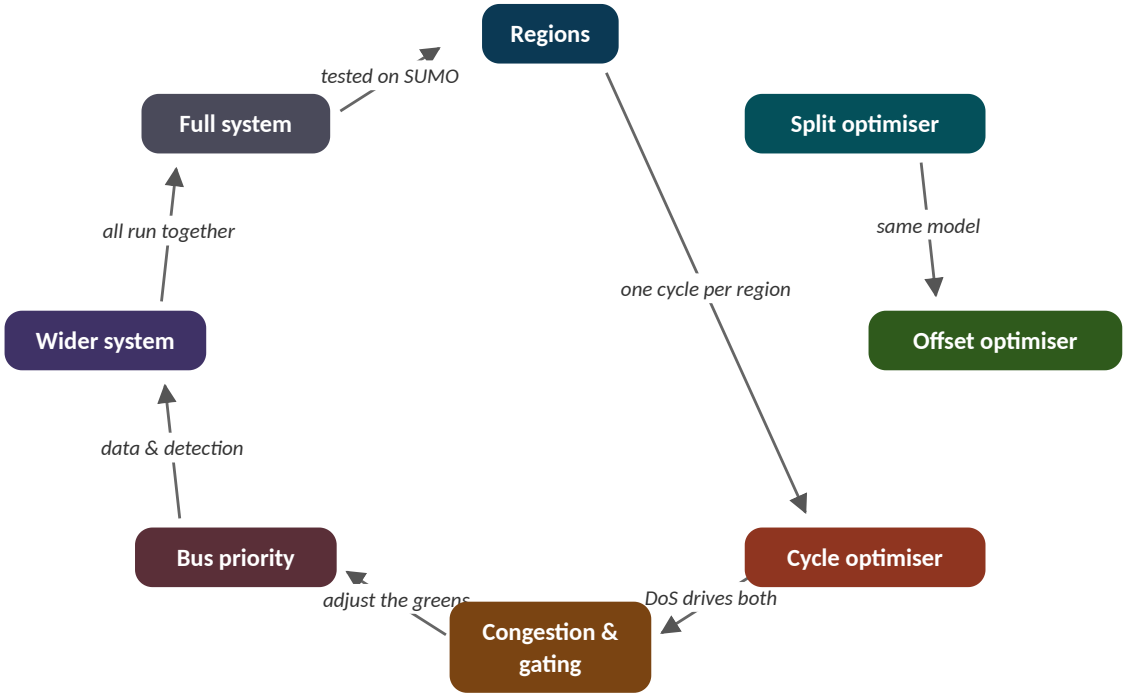


Figure 0. The whole guide on one page. Each box is one concept section; the arrows say how one idea feeds the next. Come back to this page whenever you feel lost.

1. Regions, nodes and double cycling
2. The split optimiser
3. The offset optimiser
4. The cycle optimiser
5. Congestion, exit blocking and gating
6. Bus priority: extensions and recalls
7. The wider SCOOT system
8. The full system on a SUMO region

HOW TO USE THIS GUIDE

Read one concept per sitting (20 to 30 minutes). If a word from the Foundation level feels rusty (LPU, SATO, DoS, PI), check the 'Words you will meet' table first. Do every **Simple calculation** with a pencil. Say the **Picture this** scene out loud. The **Recap so far** box grows as you go. Finish with the self-quiz and the revision plan.

What you will be able to do

- Explain what a SCOOT region is and when a node may be double cycled.
- Run the split optimiser by hand: compare −4, 0 and +4 s by the highest DoS.
- Run the offset optimiser by hand: step ±4 s towards the lowest performance index.
- Apply the cycle optimiser's rules (90% target, 4/8/16 s steps) through a peak.
- Explain congestion importance, exit blocking and gating, and when each helps.
- Choose between a bus extension and a recall, and work out the time saved.
- Name ASTRID, INGRID and the main later SCOOT features and what they do.
- Read the SUMO comparison and ablation results honestly, including the surprises.

Before we start

HOOK

Most decisions change nothing. In our SUMO run the split optimiser made 846 decisions and chose 'keep the timing' 747 times, about 88%. TRL's original description says about two thirds. Doing nothing, most of the time, is a sign the plan is already about right.

HOOK

The green wave is never designed. Nobody tells SCOOT the best offset between two junctions. It tries 4 s earlier and 4 s later once per cycle and keeps whichever its model likes best. A few minutes later the platoons arrive on green.

HOOK

Switching one thing off can help. In our ablation study, freezing the cycle at 60 s gave slightly lower delay (26.8 s) than full SCOOT (30.4 s). The reason teaches you why engineers set a maximum cycle for every region.

Words you will meet

You do not need to memorise these now. Each one is explained again when it first matters.

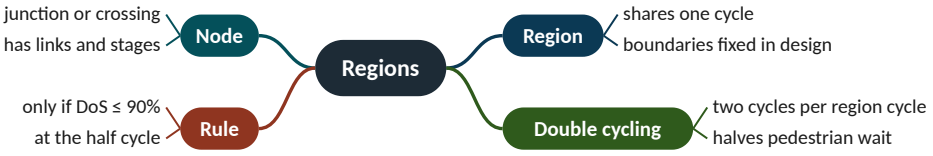
Word	In plain words
Region	A group of junctions that all use the same cycle time, so they can be coordinated.
Node	One junction or crossing inside a region.
Stage change	The moment one stage's green ends and the next stage's starts.
DoS	Degree of saturation: how full a link's green is. SCOOT aims for 90% or less.
PI	Performance index: delay + K × stops. Lower is better.
Optimiser	A small rule that tries a few options in the model and keeps the best one.
Temporary / permanent change	A change applied to this cycle only, or stored in the plan for the future.
Critical node	The busiest junction in a region; it decides the region's cycle time.
Gating	Holding traffic back on purpose, where queues do no harm, to protect a bottleneck.
AVL / SVD	Ways to detect buses: automatic vehicle location (GPS) or a selective vehicle detector.

CONCEPT 1 OF 8 · SCOOT ADVANCED

Regions, nodes and double cycling

HOOK

A busy junction needs a long cycle. A quiet pedestrian crossing next to it does not. If they must share one cycle to stay coordinated, do pedestrians have to wait as long as the busy junction's traffic?



IN PLAIN WORDS

An orchestra plays to one tempo so everyone stays together. A triangle player who has little to do can play two notes per beat and still stay in time. That is double cycling: a quiet node runs twice as fast but stays in step with its region. *Where the picture breaks:* a musician can speed up whenever they like; a node may double cycle only if its own traffic still fits into the shorter cycle.

A **region** is a group of **nodes** (junctions and crossings) that all run on one common cycle time. A shared cycle is what makes offsets meaningful: if two junctions have different cycles, their greens drift apart and no green wave can last.

Region boundaries are drawn by the engineer where roads between junctions are long enough that coordination stops mattering. Unlike SCATS, SCOOT does not join or split regions on line; the boundaries are part of the design.

A lightly loaded node, often a pedestrian crossing, can be **double cycled**: it runs two of its own cycles for every region cycle. Pedestrians wait about half as long, and the node still keeps step with its neighbours. SCOOT allows it only if the node's DoS stays at or below its target at the shorter cycle.

$$\text{average pedestrian wait} \approx (C - g)^2 \div 2C$$

for people arriving at random; g = pedestrian green (illustrative)

$$\text{double cycle allowed if DoS at } C/2 \leq 0.90$$

the node must still serve its traffic at half the cycle

SIMPLE CALCULATION · Should this crossing double cycle?

Region cycle C = 96 s. Pedestrian green g = 8 s. Vehicle demand 0.3 LPU/s, SATO 1.2 LPU/s. At C/2 = 48 s the vehicles get 30 s of effective green.

1. Wait at 96 s: $(96 - 8)^2 \div (2 \times 96) = 7744 \div 192 =$ **40.3 s**.
2. Wait at 48 s: $(48 - 8)^2 \div (2 \times 48) = 1600 \div 96 =$ **16.7 s**.
3. Vehicle demand per 48 s cycle: $0.3 \times 48 =$ **14.4 LPU**.
4. Capacity: $30 \times 1.2 =$ **36 LPU**. DoS = $14.4 \div 36 =$ **0.40**.
5. $0.40 \leq 0.90$, so double cycling is **allowed**.

Double cycling cuts the average pedestrian wait from about 40 s to about 17 s while vehicles still have plenty of spare green.

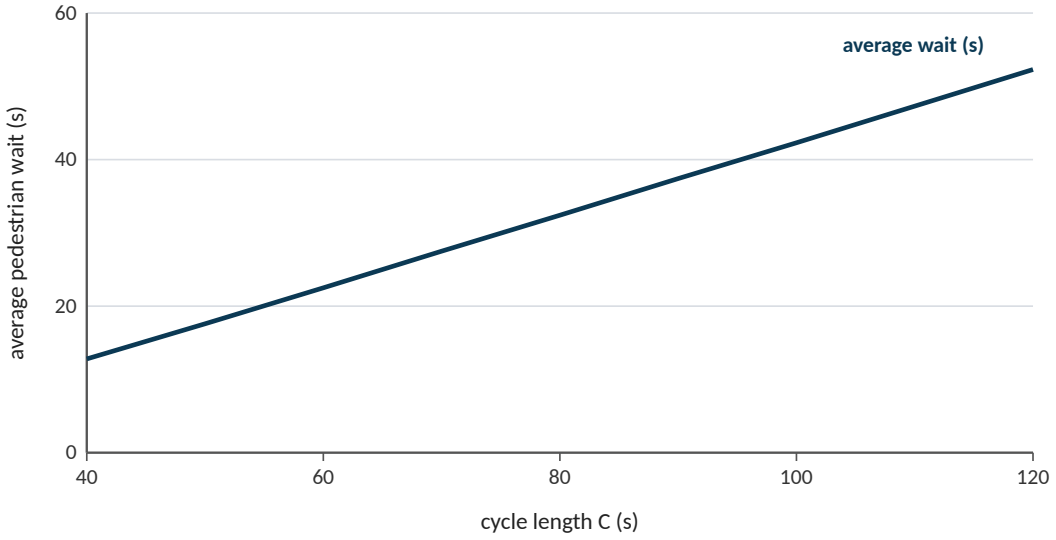


Figure 1. Average pedestrian wait against cycle length. What to notice: the wait grows almost in step with the cycle, so halving the cycle roughly halves the wait. Computed from $(C - 8)^2 \div 2C$ with an 8 s pedestrian green (illustrative).

TRAP

Double cycling does not give the crossing its own independent cycle. It runs exactly two cycles per region cycle, so it stays coordinated. A node with a truly different cycle would belong to another region.

PICTURE THIS · STOP 1 OF THE ROUTE

At the stage door the whole orchestra marches in, every foot landing on one giant metronome's tick. A tiny triangle player at the back skips two steps for every one tick, grinning, never falling out of time. *What it encodes:* One metronome = one region cycle; the marching players = nodes; the triangle player's two steps per tick = double cycling.

TRY IT NOW (about 90 seconds)

A crossing in a 120 s region has a 10 s pedestrian green. Find the average pedestrian wait at 120 s and at 60 s. *Answer in the key at the back.*

RECAP SO FAR

1. **Regions: nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.**

CONCEPT 2 OF 8 · SCOOT ADVANCED

The split optimiser

HOOK

Before every stage change SCOOT asks the simplest question imaginable: change now, 4 seconds earlier, or 4 seconds later? Why is such a tiny question enough?



IN PLAIN WORDS

Two children share a seesaw that should stay level. Every few seconds a helper may move a 4 kg bag to one side, or leave it. To remember the lesson, the helper also glues 1 kg to that side for good. *Where the picture breaks:* a seesaw balances weight; SCOOT balances the DoS of the links in the stage that is ending against the stage that is starting.

A few seconds before every scheduled stage change, the **split optimiser** tries three options in the model: make the change **4 s earlier** (the ending stage loses 4 s), **on time**, or **4 s later** (the ending stage gains 4 s). Options that would break a minimum green are skipped.

For each option it predicts the DoS of every link in the ending stage and in the starting stage. The winner is the option whose **highest** DoS is lowest. In other words, it protects the link that is closest to overflowing.

The winning move is a **temporary** change: it applies to this cycle only. A **permanent** change of 1 s in the same direction is stored in the plan, so the plan drifts slowly towards the right split without overreacting to one noisy cycle.

$$\text{score}(\text{option}) = \text{highest DoS among links in both stages}$$

pick the option with the lowest score; ties keep the timing

$$\text{DoS} = \text{demand per cycle} \div (\text{green} \times \text{SATO})$$

from the Foundation guide

SIMPLE CALCULATION · Choose −4, 0 or +4

Two stages, 60 s cycle, 50 s of green in total. Now: stage A 30 s, stage B 20 s. Demand A 24 LPU, B 10 LPU per cycle. SATO 1.0 LPU/s for both.

1. −4 (A 26, B 24): DoS A = 24 ÷ 26 = 0.92; B = 10 ÷ 24 = 0.42; highest **0.92**.
2. 0 (A 30, B 20): A = 24 ÷ 30 = 0.80; B = 10 ÷ 20 = 0.50; highest **0.80**.
3. +4 (A 34, B 16): A = 24 ÷ 34 = 0.71; B = 10 ÷ 16 = 0.63; highest **0.71**.
4. Lowest highest DoS: **+4**. A runs 34 s this cycle.
5. Permanent change: stored green A goes from 30 to **31 s**.

Stage A is the busier one, so SCOOT gives it 4 s more now and 1 s more for the future. If demand stays, the stored plan keeps creeping towards balance.

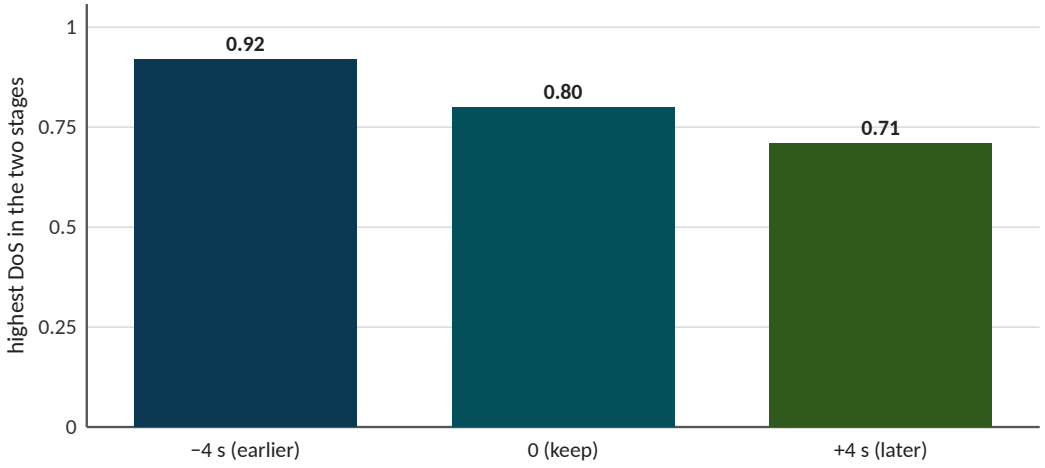


Figure 2. Highest DoS for each option in the worked example. What to notice: the optimiser does not look for the lowest delay here, only for the option that keeps the worst link furthest from overflowing. Values from the worked example.

TRAP

'+4' does not mean 'add 4 s to every stage'. It moves one stage change 4 s later: the ending stage gains 4 s and the starting stage loses 4 s, so the cycle length stays the same.

PICTURE THIS · STOP 2 OF THE ROUTE

In the green room two singers sit on a seesaw. A stagehand stands ready with a 4 kg sandbag; every few seconds he drops it on one end, or shrugs and keeps it. Each time he also glues a single 1 kg weight to that end for ever.

What it encodes: The 4 kg bag = the ±4 s temporary change; the glued 1 kg = the ±1 s permanent change; the shrug = 'keep', the most common decision.

TRY IT NOW (about 90 seconds)

Same junction, but demand is A 18 LPU and B 16 LPU per cycle (A 30 s, B 20 s now). Which option wins, and what happens to the stored green A? *Answer in the key at the back.*

RECAP SO FAR

1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser: before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.**

CONCEPT 3 OF 8 · SCOOT ADVANCED

The offset optimiser

HOOK

A green wave needs each junction to turn green just as the platoon from the previous one arrives. SCOOT finds that moment without ever being told the distance or the speed. How?



IN PLAIN WORDS

Imagine finding the lowest point of a valley in thick fog. You can only feel the ground one step to the left and one step to the right. You step towards whichever is lower, again and again, until both sides are higher. You are at the bottom. *Where the picture breaks:* a valley may have two dips; small steps can get stuck in the shallower one. Engineers can set starting offsets to avoid this.

The **offset** is when a node's cycle starts compared with the region's reference. Once per cycle, at a fixed point in each node's cycle, the **offset optimiser** asks: should this whole node run 4 s earlier, on time, or 4 s later?

For each option SCOOT re-runs its queue model on every link **into** the node and every link **out** of it to a neighbour, using the cyclic flow profiles from the Foundation guide. It adds up the **performance index** (PI = delay + K × stops) and keeps the option with the lowest total.

Because the step is small and taken every cycle, the offset 'walks downhill' on the PI curve to a good coordination and keeps tracking it as traffic changes. In our worked example the platoon takes about 22 s to travel 300 m, and the best offset turns out to be about 20 s.

$$\mathrm{PI}(\text{offset}) = \Sigma_{\text{links in and out}}\,(\text{delay} + \mathrm{K} \times \text{stops})$$

K = 4 in this lab

$$\text{new offset} = \text{best of } \{\ \text{offset} - 4,\ \text{offset},\ \text{offset} + 4\ \}$$

one step per cycle

SIMPLE CALCULATION · Walking down the PI curve

Two nodes 300 m apart (cruise time 22 s), 60 s cycle. The model's PI at each offset is: 4 s → 289, 8 s → 230, 12 s → 167, 16 s → 112, 20 s → 94, 24 s → 108. The node starts at 8 s.

1. Cycle 1, at 8 s: options 4/8/12 give 289/230/167. Lowest: **12 s**.
2. Cycle 2, at 12 s: 230/167/112. Move to **16 s**.
3. Cycle 3, at 16 s: 167/112/94. Move to **20 s**.
4. Cycle 4, at 20 s: 112/94/108. The middle is lowest: **keep 20 s**.

In four cycles (four minutes) the offset found the bottom of the curve, and PI fell from 230 to 94. SCOOT never needed to know the distance or the speed.

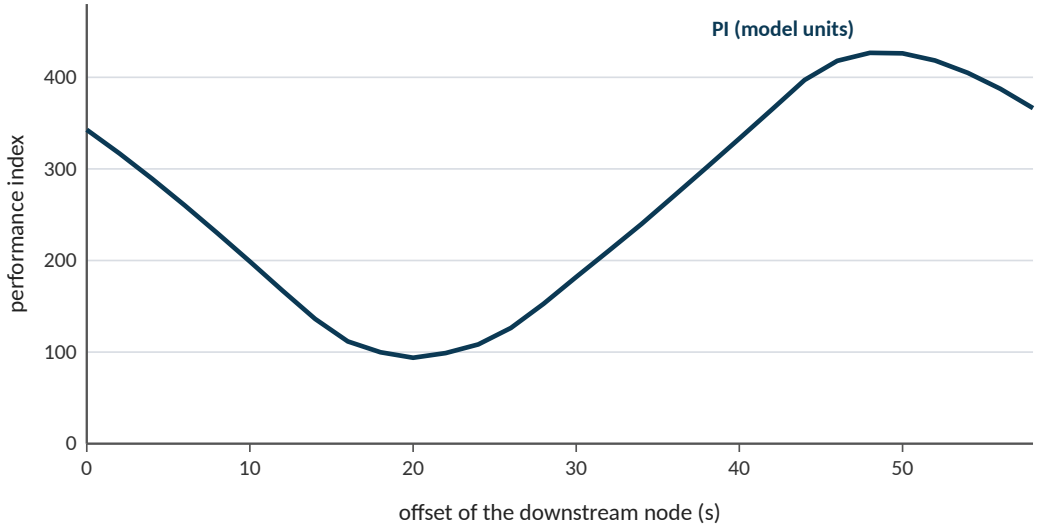


Figure 3. Model PI for every possible offset between two nodes. What to notice: one clear valley near 20 s, about the platoon's travel time. SCOOT only ever looks 4 s either side of where it is. Computed with this lab's Robertson dispersion and queue model (0.6 LPU/s platoon, 30 s greens, SATO 1.0, K = 4).

TRAP

The offset optimiser uses PI (delay and stops), not DoS. DoS hardly changes with offset, because offset moves when the green happens, not how long it is.

PICTURE THIS · STOP 3 OF THE ROUTE

In a long corridor two drummers try to make their beats meet in the middle. Each cycle the second drummer shuffles 4 steps forward or back, watching a glowing noise meter on the wall, and stops when the meter is at its lowest.

What it encodes: The drummers = two nodes; shuffling 4 steps = the ±4 s offset change; the noise meter = PI; stopping at the lowest = the valley of the PI curve.

TRY IT NOW (about 90 seconds)

Using the PI values in the worked example, where does the offset end up if it starts at 24 s? *Answer in the key at the back.*

RECAP SO FAR

1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser:** before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.
3. **Offset optimiser: once per cycle per node, try −4/0/+4 s, pick the lowest PI on links in and out; walks downhill to a green wave.**

CONCEPT 4 OF 8 · SCOOT ADVANCED

The cycle optimiser

HOOK

Longer cycles give more capacity but make everyone wait longer at red. How does SCOOT decide when the whole region should switch to a longer or shorter cycle?



IN PLAIN WORDS

A conductor sets the tempo for the whole orchestra. If the busiest section cannot fit its notes in, the conductor slows down a little. If everyone has time to spare, the conductor speeds up a little. The tempo changes only every few minutes, so the players are never confused. *Where the picture breaks:* a slower tempo in music is just slower; a longer traffic cycle adds capacity because the lost time between stages is a smaller share of each cycle.

Every 5 minutes (every 2.5 minutes while the cycle is going up) the **cycle optimiser** reviews the whole region. It finds the **critical node**: the junction whose busiest link needs the longest cycle.

The rule is simple. If the critical node's DoS is above about **90%**, the cycle goes up. If every link is comfortably below target (this lab uses **80%**), the cycle comes down. In between it holds. Steps are **4, 8 or 16 s**, bigger at longer cycles, always within the region's minimum and maximum.

Why does a longer cycle help? Each stage change wastes some lost time L. In a cycle C only C − L seconds are useful green. A longer cycle makes L a smaller share, so the same flow ratio Y gives a lower DoS.

$$\text{critical DoS} \approx Y \times C \div (C - L)$$

Y = sum of critical flow ratios, L = lost time per cycle

$$\text{step} = 4\text{ s } (C < 64),\ 8\text{ s } (C < 128),\ 16\text{ s } (C \geq 128)$$

up if DoS > 0.90, down if all < 0.80

SIMPLE CALCULATION · A rising peak

Critical node: lost time L = 12 s, flow ratio Y = 0.78 in the peak. The cycle is 40 s.

1. DoS = 0.78 × 40 ÷ (40 − 12) = 31.2 ÷ 28 = **1.11**. Above 0.90 → up 4 s to **44 s**.
2. 2.5 min later: 0.78 × 44 ÷ 32 = **1.07**. Still above → **48 s**.
3. At 64 s: 0.78 × 64 ÷ 52 = **0.96**. The step is now 8 s → **72 s**.
4. At 80 s: 0.78 × 80 ÷ 68 = **0.92** → **88 s**.
5. At 88 s the peak eases (Y = 0.70): 0.70 × 88 ÷ 76 = **0.81**. Between 0.80 and 0.90 → **hold**.

Going up takes small steps every 2.5 minutes. Coming down happens only every 5 minutes, so the region can spend a while on a cycle longer than it needs.

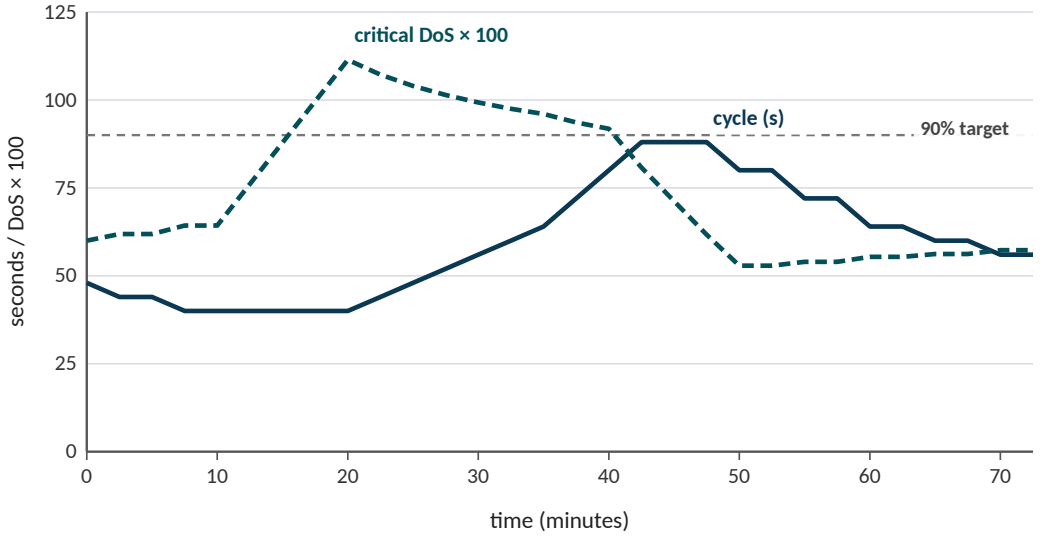


Figure 4. The cycle optimiser through a peak. What to notice: DoS shoots above 100 as demand rises; the cycle climbs in 4 s and then 8 s steps until DoS is back under 90; after the peak it falls slowly. Simulated with this lab's rules (L = 12 s, Y from 0.45 to 0.78 and back).

TRAP

A longer cycle is not always better. It lowers DoS but raises delay for everyone waiting at red. That is why the lab's optimiser also compares PI and why every region has a maximum cycle.

PICTURE THIS · STOP 4 OF THE ROUTE

On the podium stands a metronome as tall as a giraffe. Three weights hang on it, stamped 4, 8 and 16. Every five minutes a cuckoo pops out; the conductor glances at a gauge painted red above 90 and slides one weight up or down.

What it encodes: Metronome = region cycle; weights = 4/8/16 s steps; cuckoo every five minutes = review every 5 min; the gauge = critical node DoS with its 90% target.

TRY IT NOW (about 90 seconds)

A region runs a 56 s cycle. The critical node has Y = 0.70 and L = 12 s. What does the cycle optimiser do? *Answer in the key at the back.*

RECAP SO FAR

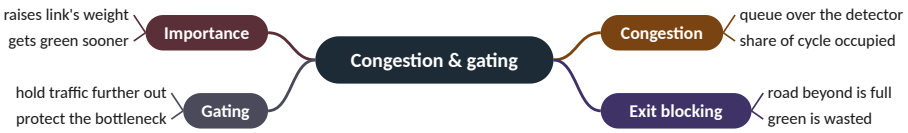
1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser:** before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.
3. **Offset optimiser:** once per cycle per node, try −4/0/+4 s, pick the lowest PI on links in and out; walks downhill to a green wave.
4. **Cycle optimiser: every 5 min (2.5 rising) the critical node's DoS decides: > 90% up, all < 80% down, steps 4/8/16 s.**

CONCEPT 5 OF 8 · SCOOT ADVANCED

Congestion, exit blocking and gating

HOOK

When a queue grows back over SCOOT's detector, the detector just sits there 'occupied'. It can no longer count arrivals. What does SCOOT do when its own eyes are covered?



IN PLAIN WORDS

A cloakroom with a sensor at the door works fine until the queue for coats reaches the door and sits on the sensor. Then the staff must notice the queue itself. If the cloakroom is about to jam completely, an usher closes the outer door for a moment so the people inside can move. *Where the picture breaks:* people in a queue can be asked to wait outside politely; cars held at a gate still need somewhere harmless to queue, so gates are chosen where queues do no damage.

When a queue reaches back over the upstream detector, the detector stays occupied by standing traffic. SCOOT measures **congestion** as the share of the cycle the detector is covered in this way.

The engineer gives each link a **congestion importance** factor. It raises that link's weight in the split optimiser, so a congested link gets green before its queue blocks the junction behind it. Later SCOOT versions also detect **exit blocking**: a link cannot discharge because the road beyond it is full, so giving it green is wasted.

Gating protects a critical bottleneck. When a critical link's DoS passes a threshold, SCOOT shortens the green on chosen **gate links** further out, where queues do no harm. The bottleneck keeps flowing instead of locking up.

$$\text{congestion f} = \text{seconds detector blocked by queue} \div \text{cycle}$$

0 = free, 1 = blocked all cycle

$$\text{weighted DoS} = \text{DoS} \times (1 + \text{importance} \times \text{f})$$

used by the split optimiser in this lab

SIMPLE CALCULATION · A congested link asks for green

A link has DoS 0.85. In a 60 s cycle its detector is covered by a standing queue for 12 s. Congestion importance = 1.0.

1. Congestion f = 12 ÷ 60 = **0.20**.
2. Weight = 1 + 1.0 × 0.20 = **1.20**.
3. Weighted DoS = 0.85 × 1.20 = **1.02**.
4. Plain DoS 0.85 was under 0.90; weighted 1.02 is **above** it.

The split optimiser now treats this link as the worst one and moves green towards it, before its queue spills back into the junction upstream.

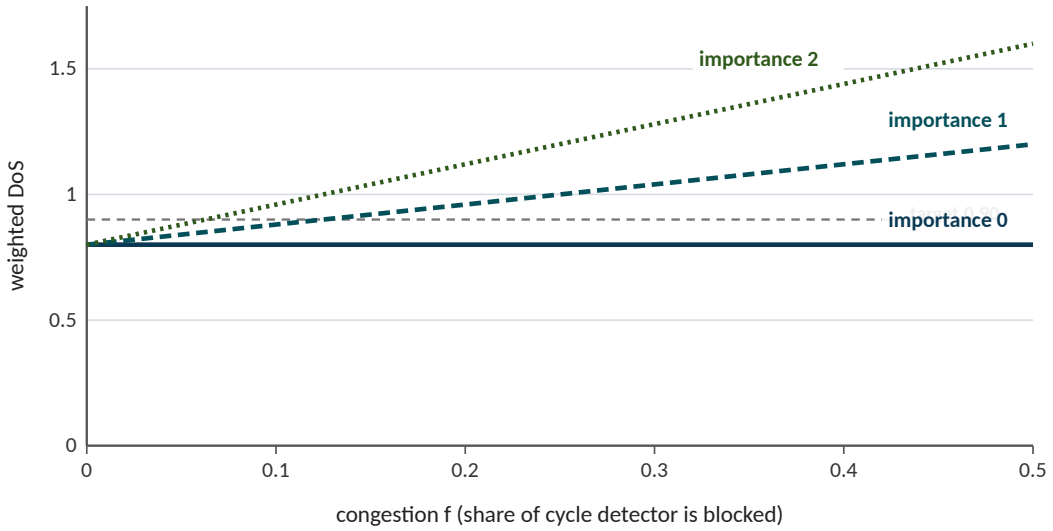


Figure 5. Weighted DoS as congestion grows (plain DoS 0.80). What to notice: with importance 0 congestion is ignored; with importance 1 or 2 the link crosses the 90% target at f ≈ 0.13 or 0.06 and wins green. Computed from DoS × (1 + importance × f).

TRAP

Gating deliberately makes some drivers wait longer. It is worth it only if the queue it creates sits somewhere harmless (a long link) and the bottleneck it protects would otherwise lock up and lose capacity for everyone.

PICTURE THIS · STOP 5 OF THE ROUTE

In the foyer a mountain of coats has piled up over the doorway's motion sensor, which flashes 'BUSY' forever. An usher in white gloves swings the outer gate shut, and outside on the long lawn concert-goers queue happily with picnic blankets.
What it encodes: Coats on the sensor = queue over the detector (congestion); the gate = gating; the long lawn = the harmless gate link where queues may wait.

TRY IT NOW (about 90 seconds)

A link has DoS 0.75. Its detector is blocked for 18 s of a 60 s cycle; importance 1.0. Is its weighted DoS above the 90% target? *Answer in the key at the back.*

RECAP SO FAR

1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser:** before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.
3. **Offset optimiser:** once per cycle per node, try −4/0/+4 s, pick the lowest PI on links in and out; walks downhill to a green wave.
4. **Cycle optimiser:** every 5 min (2.5 rising) the critical node's DoS decides: > 90% up, all < 80% down, steps 4/8/16 s.
5. **Congestion and gating: queues over the detector raise a link's weight (DoS × (1 + importance × f)); exit blocking wastes green; gating holds traffic where queues are harmless.**

CONCEPT 6 OF 8 · SCOOT ADVANCED

Bus priority: extensions and recalls

HOOK

A bus carries 50 people; a car carries one or two. If a bus arrives just as the light turns red, should the light wait for it?



IN PLAIN WORDS

A host holding a lift door for someone running down the corridor is an extension. Pressing 'close' early to fetch someone waiting on another floor is a recall. Either way, the lift then gets back to its normal routine. *Where the picture breaks:* a lift serves one person at a time; a held green also serves every car behind the bus, and the cross street pays for it.

Buses are detected by **automatic vehicle location (AVL)**, using GPS, or by a **selective vehicle detector (SVD)** that only buses trigger. SCOOT then has two tools.

An **extension** holds the current green a few seconds so a bus about to arrive gets through. A **recall** brings the bus's stage in early by cutting the current stage short, but never below its minimum green.

After either, SCOOT **recovers**: the following stages are adjusted so the plan returns to its optimised timings within a cycle or two. The authority given to buses can be weighted by how important priority is on that route. In our SUMO region, bus priority cut bus delay from 34.7 s to 22.9 s.

$$\text{extension possible if arrival} \le \text{green end} + \text{maximum extension}$$

lab maximum extension 12 s

$$\text{bus saving} = \text{wait without priority} - \text{wait with priority}$$

the cross street loses about the same time

SIMPLE CALCULATION · Extension or recall?

Cycle 80 s. Stage A (bus road) green 0-40 s, intergreen 5 s, stage B green 45-75 s, intergreen 5 s. Max extension 12 s. Minimum green for B 7 s.

1. Bus 1 reaches the stop line at 46 s. Without priority it waits until A returns at 80 s: **34 s**.
2. 46 ≤ 40 + 12 = 52, so extend A to 46 s: wait **0 s**. Saving 34 s.
3. Bus 2 is detected at 56 s and will arrive at 65 s. Without priority: 80 − 65 = **15 s**.
4. Recall: B has had 11 s (≥ 7 s minimum), so cut it at 56 s; after the 5 s intergreen A is green at 61 s.
5. Bus 2 arrives at 65 s on green: wait **0 s**. Saving 15 s.

Extensions help buses that just miss the green; recalls help buses that arrive mid-red. Stage B lost 6 s and 19 s respectively, which SCOOT recovers over the next cycles.

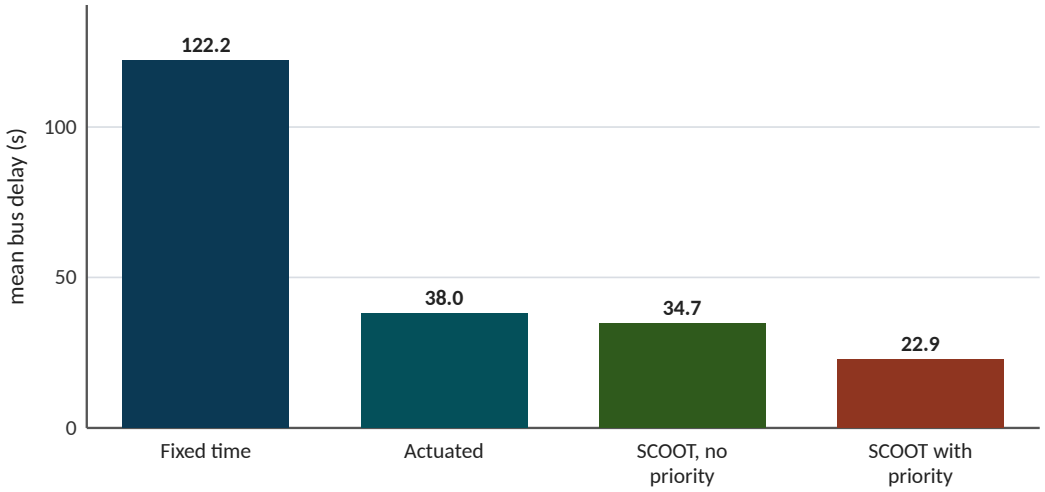


Figure 6. Mean bus delay on the SUMO region (Lab A1 and A2). What to notice: SCOOT's priority cut bus delay by about a third compared with SCOOT without priority, and to under a fifth of fixed time. From this repository's runs, seed 1.

TRAP

Priority is not free. Every second given to the bus stage is taken from another stage this cycle. That is why extensions are capped, recalls respect minimum greens, and recovery restores the plan quickly.

PICTURE THIS · STOP 6 OF THE ROUTE

A red double-decker bus rolls into the orchestra pit. The conductor holds the last violin note on and on until the bus has squeezed past (extension), then later cuts a song short with a crash of cymbals to let a second bus in (recall).

What it encodes: Held note = extension (max 12 s here); song cut short = recall after minimum green; the bus = a detected bus via GPS or a bus-only detector.

TRY IT NOW (about 90 seconds)

Same junction. A bus will reach the stop line at 50 s. Which tool works, and how much bus delay does it save? *Answer in the key at the back.*

RECAP SO FAR

1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser:** before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.
3. **Offset optimiser:** once per cycle per node, try −4/0/+4 s, pick the lowest PI on links in and out; walks downhill to a green wave.
4. **Cycle optimiser:** every 5 min (2.5 rising) the critical node's DoS decides: > 90% up, all < 80% down, steps 4/8/16 s.
5. **Congestion and gating:** queues over the detector raise a link's weight (DoS × (1 + importance × f)); exit blocking wastes green; gating holds traffic where queues are harmless.
6. **Bus priority: detect by AVL or SVD; extend green (≤ 12 s here) or recall the bus stage after minimum green; recover in 1-2 cycles. Lab: 34.7 → 22.9 s.**

CONCEPT 7 OF 8 · SCOOT ADVANCED

The wider SCOOT system

HOOK

SCOOT does more than time lights. It remembers what traffic usually looks like, notices when today looks wrong, and estimates the pollution on every street. What is it doing with all that data?



IN PLAIN WORDS

A good doctor keeps your records. When you come in with a temperature, she compares it with your normal readings, not with someone else's. ASTRID is the records cabinet; INGRID is the doctor who says 'this is not normal for you'. *Where the picture breaks:* a patient can describe symptoms; SCOOT has only detector counts and occupancies, so INGRID must infer incidents from patterns.

ASTRID is a database that stores SCOOT's data over days and weeks. Its **historic profiles** say what flow normally looks like at this link, on this kind of day, at this time. When a detector fails, SCOOT can substitute the historic profile instead of flying blind.

INGRID is real-time incident detection. It compares what the detectors see now with what ASTRID says is normal. A sudden drop in flow with very high occupancy downstream, for example, suggests a crash or a blocked lane, and an operator is alerted.

Later versions added emissions estimates (from version 4, 1997), better behaviour in oversaturation (version 5), pedestrian and low-flow features (version 6, 2010) and, in SCOOT 7 (2020), cooperative-vehicle data, pedestrian SCOOT, loop-failure logic and a cloud-first design. Some cities are now moving to newer systems such as Transport for London's FUSION, built on the same principles.

$$\text{ratio} = \text{flow now} \div \text{historic flow for this time}$$

far below 1 with high occupancy → possible incident (illustrative rule)

SIMPLE CALCULATION · Does today look normal?

At 08:15 on a weekday, ASTRID's historic profile for a link says 60 LPU per 5 minutes. Today the detector shows 12 LPU, and the detector is occupied 85% of the time (normally 20%).

1. Flow ratio = 12 ÷ 60 = **0.20**: one fifth of normal flow.
2. Occupancy ratio = 85 ÷ 20 = **4.25** times normal.
3. Low flow plus high occupancy means traffic is standing, not absent.
4. This pattern is typical of a blockage downstream: **raise an incident alert**.

Low flow alone could mean an empty road at night. Low flow with very high occupancy means a queue that is not moving. Comparing with history is what makes the difference visible.

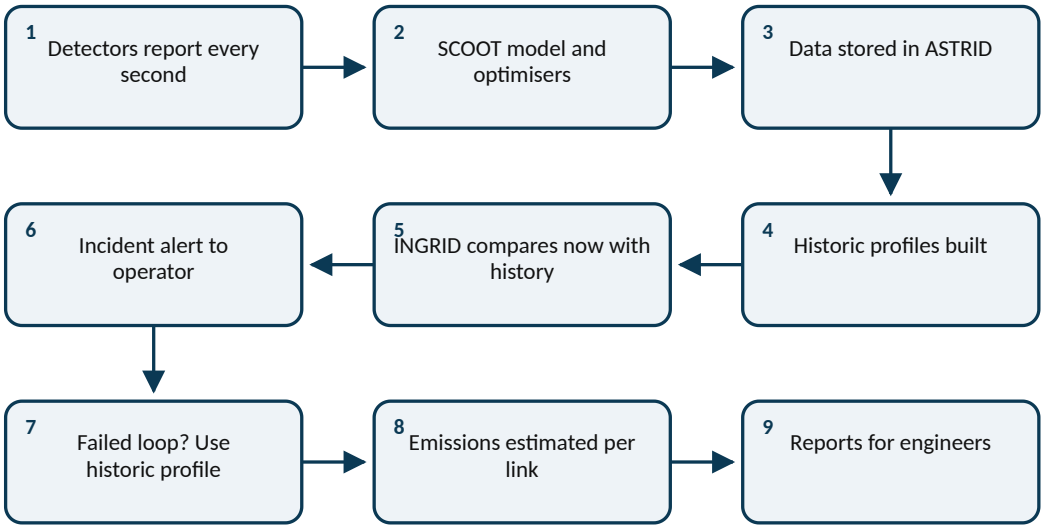


Figure 7. How the wider system uses SCOOT's data. What to notice: the same detector data feed three jobs: control, memory (ASTRID) and alarms (INGRID). Read left to right, then down.

TRAP

Historic substitution is a fallback, not a fix. A link running on yesterday's profile cannot react to today's surprise, so failed detectors must still be repaired quickly. (Mastery tests exactly this.)

PICTURE THIS · STOP 7 OF THE ROUTE

Behind the stage is a dusty archive where an archivist named Astrid files a recording of every concert. Her twin sister Ingrid listens to tonight's music through a brass trumpet and rings a fire bell the moment it sounds unlike the recordings.

What it encodes: Astrid's archive = the ASTRID database of historic profiles; Ingrid's bell = INGRID incident detection comparing now with history.

TRY IT NOW (about 90 seconds)

A link's history says 40 LPU per 5 min at 02:00. Tonight it reads 8 LPU and occupancy is 3% (normally 4%). Incident or not? *Answer in the key at the back.*

RECAP SO FAR

1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser:** before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.
3. **Offset optimiser:** once per cycle per node, try −4/0/+4 s, pick the lowest PI on links in and out; walks downhill to a green wave.
4. **Cycle optimiser:** every 5 min (2.5 rising) the critical node's DoS decides: > 90% up, all < 80% down, steps 4/8/16 s.
5. **Congestion and gating:** queues over the detector raise a link's weight (DoS × (1 + importance × f)); exit blocking wastes green; gating holds traffic where queues are harmless.
6. **Bus priority:** detect by AVL or SVD; extend green (≤ 12 s here) or recall the bus stage after minimum green; recover in 1-2 cycles. Lab: 34.7 → 22.9 s.
7. **Wider system: ASTRID stores historic profiles (and backs up failed loops), INGRID flags incidents by comparing now with history; later versions added emissions, pedestrians and cooperative data.**

CONCEPT 8 OF 8 · SCOOT ADVANCED

The full system on a SUMO region

HOOK

We put every mechanism together on four junctions in SUMO, then switched them off one at a time. Some results were expected. Two were surprises.



IN PLAIN WORDS

To learn what each part of a car does, a mechanic removes one part at a time and test-drives it. Remove the steering and you crash; remove the radio and nothing changes. An ablation study does the same with SCOOT's mechanisms. *Where the picture breaks:* a car part is either there or not; SCOOT's mechanisms interact, so removing one can change what the others do.

Four junctions 300 m apart form one SCOOT region. Demand starts low, then a 30-minute eastbound peak, a westbound peak, and low traffic again, with buses every 4 minutes each way. All the mechanisms run together.

Lab A1 compares controllers on the same demand: mean delay **86.95 s** fixed, **29.07 s** actuated, **30.44 s** SCOOT. SCOOT had the fewest stops (1.10 per vehicle against 1.15 actuated) and much lower bus delay (22.9 against 38.0 s). On delay alone, good actuation was slightly ahead on this short corridor.

Lab A2 removes one mechanism at a time. Without the split optimiser delay more than doubles (77.3 s): a fixed 60/40 split cannot serve tidal peaks. Without offsets it rises to 39.7 s. Two surprises: freezing the cycle at 60 s did slightly better (26.8 s), because after the first peak the cycle came down only 8 s every 5 minutes; and congestion importance changed nothing, because queues never reached the detectors.

$$\text{change} = (\text{new} - \text{old}) \div \text{old} \times 100\%$$

how we compare runs

SIMPLE CALCULATION · Reading the results

Lab A1 and A2 mean delays (s per vehicle): fixed 86.95, actuated 29.07, SCOOT 30.44; no split optimiser 77.30. Bus delay: actuated 37.99, SCOOT 22.92.

1. SCOOT vs fixed: (30.44 − 86.95) ÷ 86.95 = **−65%** delay.
2. SCOOT vs actuated: (30.44 − 29.07) ÷ 29.07 = **+4.7%** delay.
3. Bus delay vs actuated: (22.92 − 37.99) ÷ 37.99 = **−40%**.
4. No split optimiser: 77.30 ÷ 30.44 = **2.5 times** the delay.

SCOOT is far better than a fixed plan, about level with good actuation on general delay, and clearly better for buses and stops. The split optimiser is the mechanism that matters most here.

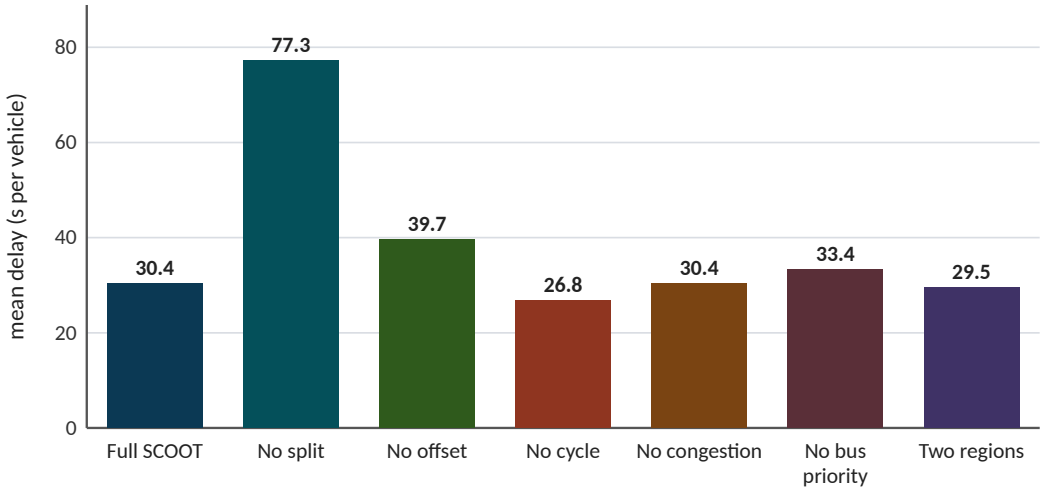


Figure 8. Ablation (Lab A2): mean delay with one mechanism removed. What to notice: removing the split optimiser hurts most; removing the cycle optimiser actually helped on this demand; congestion importance had no effect. SUMO, same demand and seed for every bar.

TRAP

Do not conclude 'the cycle optimiser is useless'. On this demand the region's maximum (120 s) and slow decreases let the cycle stay too long after the peak. With a tuned maximum it would help; that is why engineers tune each region.

PICTURE THIS · STOP 8 OF THE ROUTE

At the final bow a giant applause meter shows the night's 'grumble' scores: a clockwork robot scores 87, a jumpy rabbit 29, the conductor 30. When the violinists are sent home the grumble leaps to 77; when the metronome is glued at 60, it drops to 27.

What it encodes: Scores = Lab A1 delays (fixed 87, actuated 29, SCOOT 30 s); violins sent home = no split optimiser (77 s); glued metronome = fixed cycle (26.8 s).

TRY IT NOW (about 90 seconds)

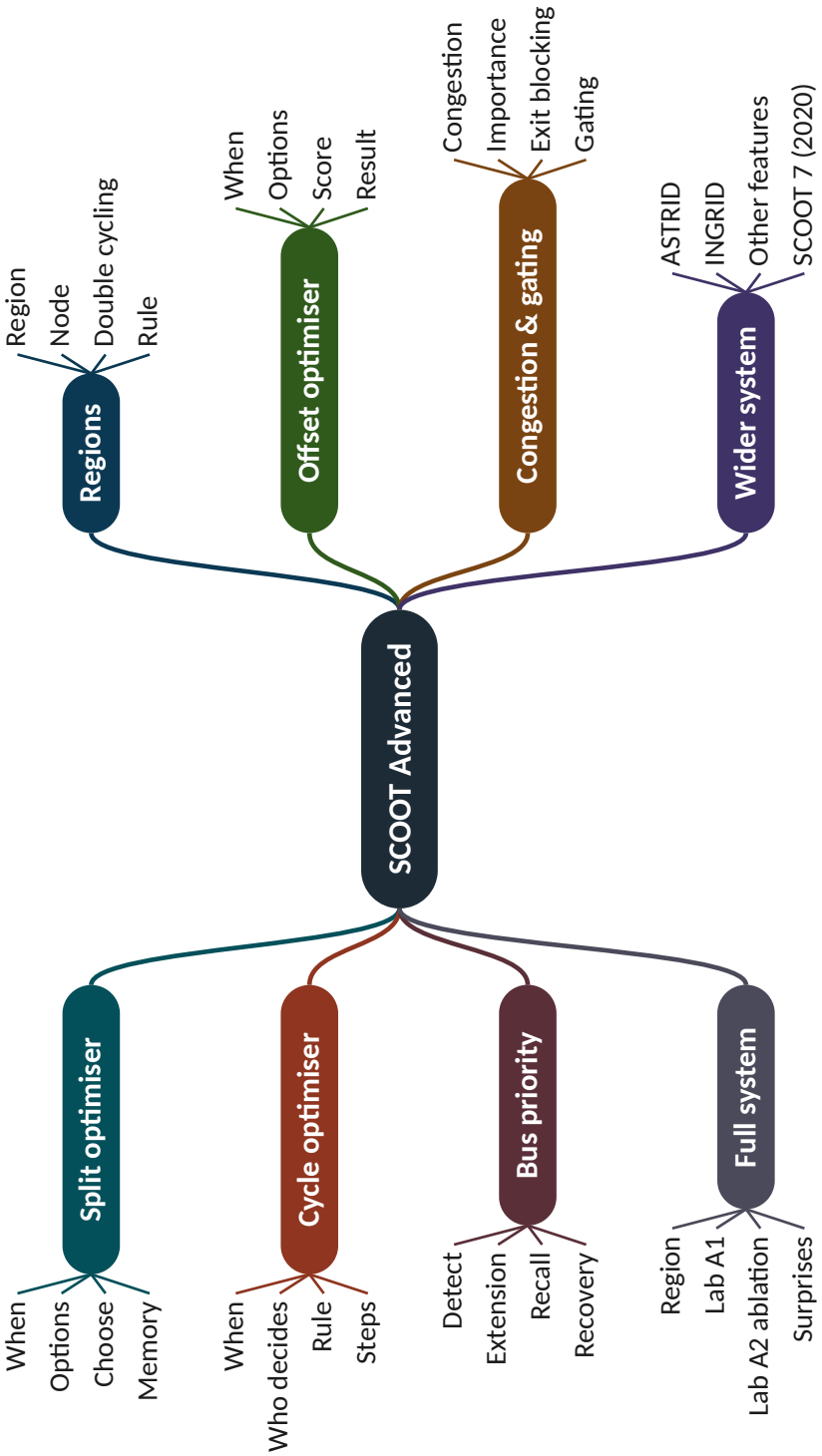
Without offset optimisation the mean delay was 39.74 s; full SCOOT 30.44 s. By what percentage does offset optimisation reduce delay? *Answer in the key at the back.*

RECAP SO FAR

1. **Regions:** nodes in a region share one cycle so offsets work; a quiet node may double cycle if its DoS stays ≤ 90% at half the cycle.
2. **Split optimiser:** before each stage change try −4/0/+4 s, pick the lowest highest DoS; temporary ±4 s, permanent ±1 s.
3. **Offset optimiser:** once per cycle per node, try −4/0/+4 s, pick the lowest PI on links in and out; walks downhill to a green wave.
4. **Cycle optimiser:** every 5 min (2.5 rising) the critical node's DoS decides: > 90% up, all < 80% down, steps 4/8/16 s.
5. **Congestion and gating:** queues over the detector raise a link's weight (DoS × (1 + importance × f)); exit blocking wastes green; gating holds traffic where queues are harmless.
6. **Bus priority:** detect by AVL or SVD; extend green (≤ 12 s here) or recall the bus stage after minimum green; recover in 1-2 cycles. Lab: 34.7 → 22.9 s.
7. **Wider system:** ASTRID stores historic profiles (and backs up failed loops), INGRID flags incidents by comparing now with history; later versions added emissions, pedestrians and cooperative data.
8. **Full system: Lab A1 delay fixed 87.0 / actuated 29.1 / SCOOT 30.4 s (bus 22.9 s); ablation: split matters most (77.3 s without); fixed cycle 26.8 s and congestion no effect are honest surprises.**

The master mind map

Every concept and its main branches on one page. Turn the page sideways. Try redrawing it from memory on blank paper, then compare.



Connections and contrasts

The three optimisers side by side

	Split	Offset	Cycle
Runs	before every stage change	once per cycle per node	every 5 min (2.5 rising)
Changes	one stage change −4/0/+4 s	whole node −4/0/+4 s	region cycle ±4/8/16 s
Chooses by	lowest highest DoS	lowest PI on links in and out	critical node DoS vs 90%
Memory	temporary ±4, permanent ±1	kept until next step	kept until next review
Without it (Lab A2)	77.3 s delay	39.7 s delay	26.8 s (surprise)

Things beginners mix up

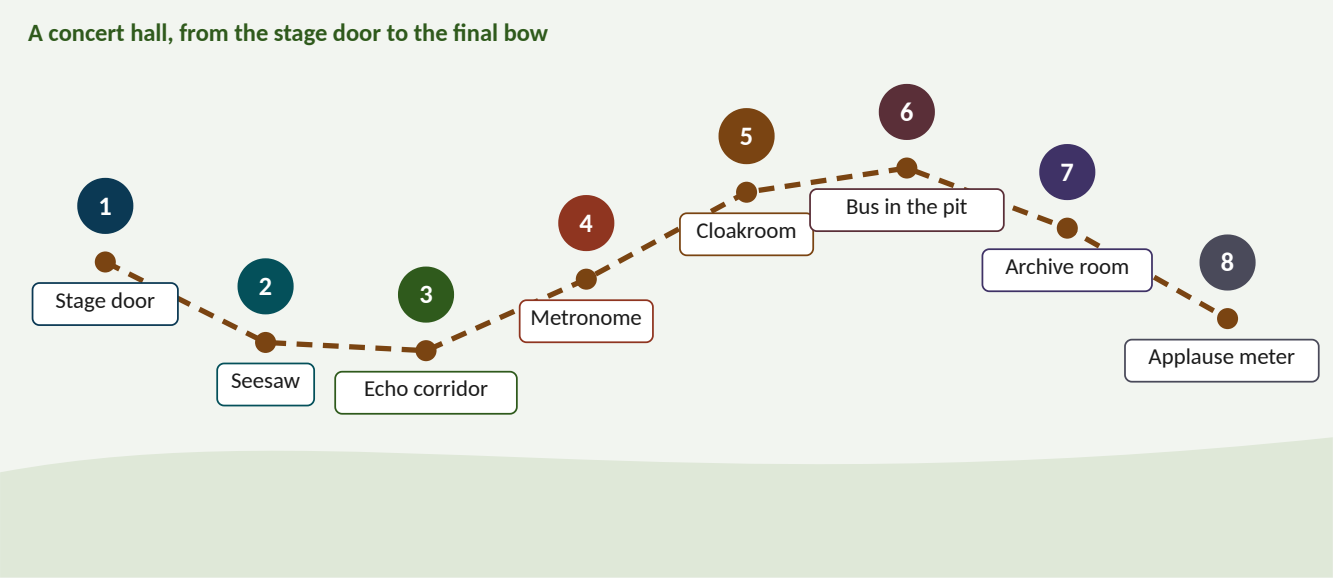
This…	…is not this	The one fact that tells them apart
Split optimiser	**Offset optimiser**	Split moves one stage change and scores by DoS; offset moves the whole node and scores by PI.
Temporary change	**Permanent change**	Temporary ±4 s applies to this cycle only; permanent ±1 s is stored in the plan.
Extension	**Recall**	Extension holds the bus's green on; recall brings the bus's stage in early.
ASTRID	**INGRID**	ASTRID stores history; INGRID compares now with history to spot incidents.
Double cycling	**A separate region**	A double-cycled node runs exactly 2 cycles per region cycle, so it stays coordinated.
Congestion	**Exit blocking**	Congestion: queue over this link's detector. Exit blocking: no room on the road beyond.

Symbols and terms

Symbol / term	Meaning
C	region cycle time (s)
L	lost time per cycle (s)
Y	sum of critical flow ratios
DoS	degree of saturation; target ≈ 0.90
PI	performance index = delay + K × stops (K = 4 here)
f	congestion: share of the cycle the detector is blocked by a queue
AVL / SVD	automatic vehicle location / selective vehicle detector
ASTRID / INGRID	SCOOT's database / incident detection

The route: A concert hall, from the stage door to the final bow

SCOOT is like an orchestra: many players (junctions) keeping one tempo (the cycle), each adjusting a little. Walk through a concert hall from the stage door to the final bow. Eight stops, one per concept. Make each scene loud and silly.



The memory route. Walk it in order; each stop holds one concept.

Stop 1: Stage door (Regions, nodes and double cycling). At the stage door the whole orchestra marches in, every foot landing on one giant metronome's tick. A tiny triangle player at the back skips two steps for every one tick, grinning, never falling out of time. *One metronome = one region cycle; the marching players = nodes; the triangle player's two steps per tick = double cycling.*

Stop 2: Seesaw (The split optimiser). In the green room two singers sit on a seesaw. A stagehand stands ready with a 4 kg sandbag; every few seconds he drops it on one end, or shrugs and keeps it. Each time he also glues a single 1 kg weight to that end for ever. *The 4 kg bag = the ±4 s temporary change; the glued 1 kg = the ±1 s permanent change; the shrug = 'keep', the most common decision.*

Stop 3: Echo corridor (The offset optimiser). In a long corridor two drummers try to make their beats meet in the middle. Each cycle the second drummer shuffles 4 steps forward or back, watching a glowing noise meter on the wall, and stops when the meter is at its lowest. *The drummers = two nodes; shuffling 4 steps = the ±4 s offset change; the noise meter = PI; stopping at the lowest = the valley of the PI curve.*

Stop 4: Metronome (The cycle optimiser). On the podium stands a metronome as tall as a giraffe. Three weights hang on it, stamped 4, 8 and 16. Every five minutes a cuckoo pops out; the conductor glances at a gauge painted red above 90 and slides one weight up or down. *Metronome = region cycle; weights = 4/8/16 s steps; cuckoo every five minutes = review every 5 min; the gauge = critical node DoS with its 90% target.*

Stop 5: Cloakroom (Congestion, exit blocking and gating). In the foyer a mountain of coats has piled up over the doorway's motion sensor, which flashes 'BUSY' forever. An usher in white gloves swings the outer gate shut, and outside on the long lawn concert-goers queue happily with picnic blankets. *Coats on the sensor = queue over the detector (congestion); the gate = gating; the long lawn = the harmless gate link where queues may wait.*

Stop 6: Bus in the pit (Bus priority: extensions and recalls). A red double-decker bus rolls into the orchestra pit. The conductor holds the last violin note on and on until the bus has squeezed past (extension), then later cuts a song short with a crash of cymbals to let a second bus in (recall). *Held note = extension (max 12 s here); song cut short = recall after minimum green; the bus = a detected bus via GPS or a bus-only detector.*

Stop 7: Archive room (The wider SCOOT system). Behind the stage is a dusty archive where an archivist named Astrid files a recording of every concert. Her twin sister Ingrid listens to tonight's music through a brass trumpet and rings a fire bell the moment it sounds unlike the recordings. *Astrid's archive = the ASTRID database of historic profiles; Ingrid's bell = INGRID incident detection comparing now with history.*

Stop 8: Applause meter (The full system on a SUMO region). At the final bow a giant applause meter shows the night's 'grumble' scores: a clockwork robot scores 87, a jumpy rabbit 29, the conductor 30. When the violinists are sent home the grumble leaps to 77; when the metronome is glued at 60, it drops to 27. *Scores = Lab A1 delays (fixed 87, actuated 29, SCOOT 30 s); violins sent home = no split optimiser (77 s); glued metronome = fixed cycle (26.8 s).*

WALK-THROUGH DRILL

Close the guide. Walk the route in your head, saying each stop out loud and the one number or formula it holds. Then check against the cheat sheet. Repeat until you can do it in under two minutes.

Self-quiz

18 questions: warm-up (recall), core (use the ideas), challenge (combine them). Answer on paper first, then check the key.

Warm-up

Q1. What do all nodes in a SCOOT region share?

A. The same split
B. The same cycle time
C. The same detectors
D. The same offset

Q2. Which options does the split optimiser compare before a stage change?

A. −10, 0, +10 s
B. −4, 0, +4 s
C. −1, 0, +1 s
D. Every possible split

Q3. How often does the offset optimiser run for a node?

A. Every second
B. Once per cycle
C. Every 5 minutes
D. Once a day

Q4. What does a bus 'extension' do?

A. Adds a bus lane
B. Holds the current green a few seconds
C. Starts the bus stage early
D. Lengthens the cycle

Q5. What is ASTRID?

A. An incident detector
B. A database of SCOOT data and historic profiles
C. A bus detector
D. A cycle optimiser

Core

Q6. Greens A 30 s, B 20 s; demand A 20 LPU, B 18 LPU; SATO 1.0. Which split option wins?

A. −4 s
B. 0
C. +4 s
D. No option is allowed

Q7. After a '+4' split decision, what happens to the stored plan?

A. Nothing
B. It changes by +1 s
C. It changes by +4 s
D. It is replaced by a new plan

Q8. What does the offset optimiser use to compare options?

A. The highest DoS
B. PI on links into and out of the node
C. Bus delay only
D. The cycle length

Q9. Offset PI values: 12 s → 167, 16 s → 112, 20 s → 94. The node is at 16 s. What does it do this cycle?

A. Move to 12 s
B. Keep 16 s
C. Move to 20 s
D. Jump to the best offset anywhere

Q10. Cycle 60 s, L = 12 s, Y = 0.75. What does the cycle optimiser do?

A. Shorten by 4 s
B. Hold
C. Lengthen by 4 s
D. Lengthen by 8 s

Q11. Why does a longer cycle lower the critical DoS for the same flows?

A. Cars drive faster
B. Lost time becomes a smaller share of the cycle
C. SATO increases
D. Detectors count fewer cars

Q12. A link with DoS 0.80 has its detector blocked 15 s of a 60 s cycle; importance 2. Weighted DoS?

A. 0.80
B. 0.90
C. 1.00
D. 1.20

Q13. What is gating for?

A. Stopping buses
B. Holding traffic where queues are harmless to protect a bottleneck
C. Closing roads at night
D. Double cycling crossings

Q14. In Lab A2, which removed mechanism raised delay most?

A. Offset optimiser
B. Cycle optimiser
C. Split optimiser
D. Bus priority

Challenge

Q15. Cycle 80 s, bus stage A green 0-40 s, max extension 12 s. A bus arrives at 55 s. Best tool?

A. Extension
B. Recall
C. Neither is possible
D. Double cycling

Q16. Freezing the cycle at 60 s gave lower delay (26.8 s) than full SCOOT (30.4 s). The best explanation?

A. The cycle optimiser has a bug
B. After the peak, slow 8 s decreases kept the cycle longer than needed
C. 60 s is always optimal
D. Buses prefer 60 s

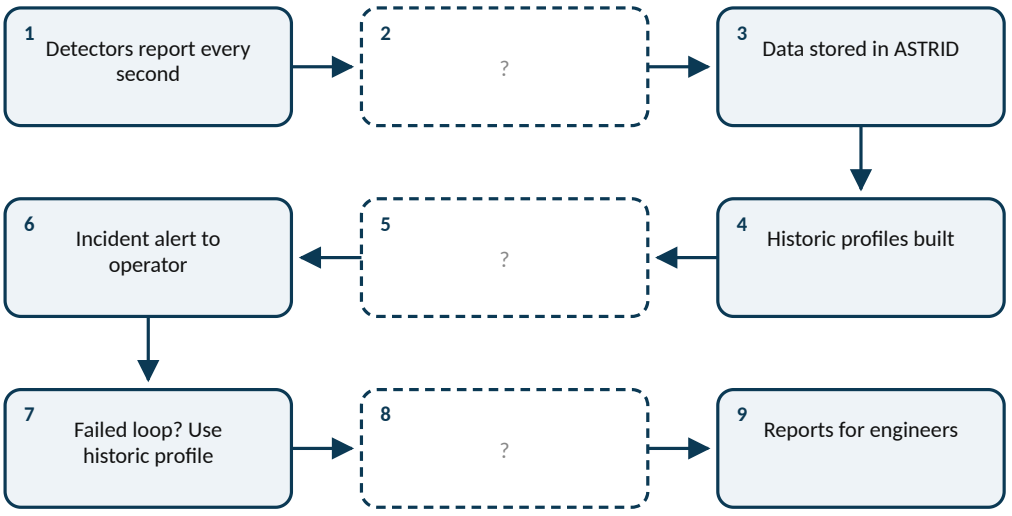
Q17. A crossing in a 100 s region wants to double cycle. At 50 s its vehicle DoS would be 0.95. Allowed?

A. Yes, pedestrians come first
B. No, DoS would exceed the 0.90 target
C. Yes, if it moves to another region
D. Only at night

Q18. Flow is 25% of the historic profile and occupancy is 4 times normal. INGRID would most likely:

A. Ignore it: low flow is normal at night
B. Flag a possible incident: traffic is standing
C. Lengthen the cycle
D. Switch the detector off

Draw it. Fill the blank boxes of this flow from memory (it is Figure 7).



Where to go if you missed a question

Missed	Do this
Q1, Q17	Re-read concept 1: Regions, nodes and double cycling
Q2, Q6, Q7	Re-read concept 2: The split optimiser
Q3, Q8, Q9	Re-read concept 3: The offset optimiser
Q10, Q11	Re-read concept 4: The cycle optimiser
Q12, Q13	Re-read concept 5: Congestion, exit blocking and gating
Q4, Q15	Re-read concept 6: Bus priority: extensions and recalls
Q5, Q18	Re-read concept 7: The wider SCOOT system
Q14, Q16	Re-read concept 8: The full system on a SUMO region

Make it stick

BUILD THE SMALLEST WORKING EXAMPLE (15 min)

On paper, run the split optimiser for five cycles on a two-stage junction: stored greens 30/20 s, SATO 1.0, demand A 24 LPU and B 10 LPU every cycle. Each cycle, score −4/0/+4 from the stored greens, apply the winner for that cycle and move the stored green A by 1 s in the same direction.

Self-check: Cycles 1-3 choose +4 (highest DoS 0.71, 0.69, 0.71) as stored A goes 30 → 31 → 32 → 33. From stored A = 33, 'keep' scores 0.73 against 0.77 for +4, so 'keep' wins from cycle 4 on: the plan has settled at 33/17 s.

TEACH IT TO A 14-YEAR-OLD (10 min)

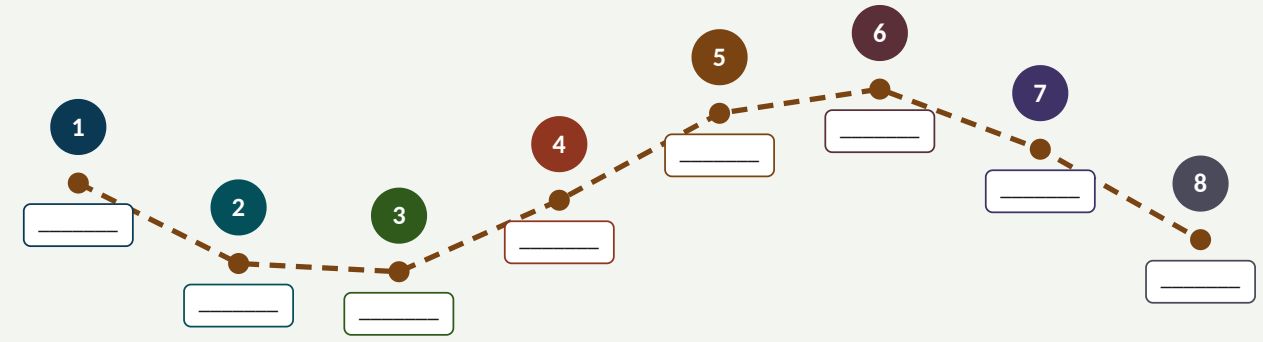
Explain to a 14-year-old how a row of traffic lights on one street can learn to give a car a 'green wave' without anyone measuring the street.

Words you may not use: optimiser, offset, performance index, saturation, region

Fill the blank route

Write the concept held at each stop, and its key number.

A concert hall, from the stage door to the final bow



One-page cheat sheet

1. Regions, nodes and double cycling

Region = nodes sharing one cycle; boundaries fixed in design (no on-line marriage, unlike SCATS). Double cycling: 2 node cycles per region cycle, only if DoS ≤ 0.9 at C/2. Ped wait ≈ $(C - g)^2/2C$: 96 s → 40.3 s, 48 s → 16.7 s.

Route: Stage door

2. The split optimiser

Before every stage change: −4 / 0 / +4 s. Score = max DoS of links in ending + starting stage; lowest wins; min greens kept. Temporary ±4, permanent ±1. Example: A 24 LPU, B 10 LPU, greens 30/20 → +4 (0.71).

Route: Seesaw

3. The offset optimiser

Once per cycle per node: node −4 / 0 / +4 s. Score = PI (delay + K·stops, K = 4 here) over links in and out. Small steps track traffic. Example 300 m, 22 s cruise: 8 → 12 → 16 → 20 s, PI 230 → 94.

Route: Echo corridor

4. The cycle optimiser

Every 5 min (2.5 rising). Critical node DoS ≈ Y·C/(C − L). > 0.90 up; all < 0.80 down (lab); else hold. Steps 4 (C < 64), 8 (< 128), 16. Within region min/max (lab 40-120 s). Example L 12, Y 0.78: 40 → 44 → … → 88 s.

Route: Metronome

5. Congestion, exit blocking and gating

Congestion f = blocked seconds / cycle. Weighted DoS = DoS·(1 + importance·f): 0.85, f 0.2, imp 1 → 1.02. Exit blocking = road beyond full. Gating: critical link over threshold → cut green on gate links further out.

Route: Cloakroom

6. Bus priority: extensions and recalls

Detection: AVL (GPS) or SVD. Extension: hold green, ≤ max (12 s lab). Recall: end other stage early, after its min green. Recovery within 1-2 cycles. Lab bus delay: fixed 122.2, actuated 38.0, SCOOT no priority 34.7, with priority 22.9 s.

Route: Bus in the pit

7. The wider SCOOT system

ASTRID = database, historic profiles, fault substitution. INGRID = incident detection, now vs history. v3.1 (1995) bus priority, ASTRID, INGRID; v4 (1997) emissions; v5 MC3 congestion; v6 (2010) pedestrian, low flow; SCOOT 7 (2020) cooperative data, cloud-first.

Route: Archive room

8. The full system on a SUMO region

Lab A1: delay 86.95 / 29.07 / 30.44 s; stops 1.77 / 1.15 / 1.10; bus 122.2 / 38.0 / 22.9 s. Lab A2: no split 77.3, no offset 39.7, no cycle 26.8, no congestion 30.4, no bus 33.4 (bus 34.7), two regions 29.5.

Route: Applause meter

Numbers to remember

- Split: −4/0/+4 s temporary, ±1 s permanent; 88% 'keep' in the lab
- Offset: once per cycle, ±4 s, PI = delay + 4 × stops
- Cycle: every 5 min (2.5 rising), > 0.90 up, < 0.80 down, steps 4/8/16 s
- Double cycling: 96 s → 48 s cuts ped wait 40.3 → 16.7 s
- Bus: max extension 12 s; bus delay 34.7 → 22.9 s
- Lab A1 delay: fixed 86.95, actuated 29.07, SCOOT 30.44 s
- Lab A2: no split 77.3, no offset 39.7, no cycle 26.8 s

Answer key

Try it now

Try it 1 (Regions, nodes and double cycling): At 120 s: $(120 - 10)^2 \div 240 = 12100 \div 240 =$ **50.4 s**. At 60 s: $(60 - 10)^2 \div 120 = 2500 \div 120 =$ **20.8 s**.

Try it 2 (The split optimiser): −4: 18/26 = 0.69, 16/24 = 0.67 → 0.69. 0: 0.60, 0.80 → 0.80. +4: 18/34 = 0.53, 16/16 = 1.00 → 1.00. **−4 wins** (change 4 s earlier); stored green A drops to **29 s**.

Try it 3 (The offset optimiser): At 24: options 20/24/28 give 94/108/153 → move to **20 s**. At 20: 112/94/108 → **keep 20 s**.

Try it 4 (The cycle optimiser): DoS = 0.70 × 56 ÷ 44 = 39.2 ÷ 44 = **0.89**. That is between 0.80 and 0.90, so it **holds** at 56 s.

Try it 5 (Congestion, exit blocking and gating): f = 18 ÷ 60 = 0.30; weighted DoS = 0.75 × 1.30 = **0.975**. Yes, above 0.90, so it will be favoured.

Try it 6 (Bus priority: extensions and recalls): 50 ≤ 52 (40 + 12), so an **extension** to 50 s works. Without priority the bus waits 80 − 50 = 30 s, so it saves **30 s**.

Try it 7 (The wider SCOOT system): Flow ratio 8 ÷ 40 = 0.2, but occupancy is low and about normal (3/4 = 0.75). Traffic is light and moving, not stuck: **probably no incident** (perhaps a quiet night or a diversion).

Try it 8 (The full system on a SUMO region): (39.74 − 30.44) ÷ 39.74 = 9.30 ÷ 39.74 = **23%** less delay with offset optimisation.

Self-quiz

Q1: B. A common cycle makes offsets meaningful. Splits and offsets differ node by node. (Concept 1)

Q2: B. ±4 s temporary; ±1 s is the permanent change, not the options compared. (Concept 2)

Q3: B. Once per cycle per node. Every 5 minutes is the cycle optimiser. (Concept 3)

Q4: B. Starting the bus stage early is a recall. (Concept 6)

Q5: B. INGRID detects incidents; ASTRID stores the data and profiles. (Concept 7)

Q6: A. −4: 20/26 = 0.77, 18/24 = 0.75 → 0.77. 0: 0.67, 0.90 → 0.90. +4: 0.59, 1.13 → 1.13. Lowest highest DoS is −4. (Concept 2)

Q7: B. The +4 s applies this cycle only; the plan drifts by 1 s in the same direction. (Concept 2)

Q8: B. Offset changes when green happens, not its length, so DoS barely changes; PI (delay + K·stops) does. (Concept 3)

Q9: C. Of 12/16/20, 20 s has the lowest PI (94). SCOOT only looks ±4 s. (Concept 3)

Q10: C. DoS = 0.75 × 60 ÷ 48 = 0.94 > 0.90, and 60 < 64 so the step is 4 s → 64 s. (Concept 4)

Q11: B. Useful green is C − L; the ratio C/(C − L) falls as C grows. (Concept 4)

Q12: D. f = 0.25; 0.80 × (1 + 2 × 0.25) = 0.80 × 1.5 = 1.20. (Concept 5)

Q13: B. Gating trims green on gate links further out so a critical link does not lock up. (Concept 5)

Q14: C. No split optimiser: 77.3 s against 30.4 s for full SCOOT. (Concept 8)

Q15: B. 55 > 40 + 12 = 52, so an extension cannot reach; a recall (after B's minimum green) can bring A back early. (Concept 6)

Q16: B. The cycle rose to 88 s and came down only 8 s per 5 minutes; tuning the region's maximum fixes this. (Concept 8)

Q17: B. Double cycling is allowed only if the node stays at or below its target at the shorter cycle. (Concept 1)

Q18: B. Low flow with high occupancy means vehicles are stuck on the detector, not absent. (Concept 7)

Draw it: the completed flow is Figure 7.

Your revision plan

When	What	Done
Today	Read one concept a sitting. Do every Try-it-now on paper.	☐
Day 1	Walk the route from memory. Redraw the master mind map.	☐
Day 3	Warm-up and core quiz without notes. Re-read what you missed.	☐
Day 7	Whole quiz. Do the 'build the smallest example' task.	☐
Day 14	Teach-it task out loud. Cheat sheet from memory, then compare.	☐
Before an exam or project	Cheat sheet plus route walk only.	☐

Going further

- Hunt, P.B., Robertson, D.I., Bretherton, R.D. and Winton, R.I. (1981). *SCOOT: a traffic responsive method of coordinating signals*. TRRL Laboratory Report LR1014.
- Bretherton, R.D. (1989). SCOOT urban traffic control system: philosophy and evaluation. *IFAC Control, Computers, Communications in Transportation*.
- Robertson, D.I. and Bretherton, R.D. (1991). Optimizing networks of traffic signals in real time: the SCOOT method. *IEEE Transactions on Vehicular Technology* 40(1), 11-15.
- This lab's website: Advanced page and SUMO labs A1-A2, smrahmanbd.github.io/scoot-learning-lab.

HONESTY NOTE

Worked examples use small, invented numbers chosen to be easy by hand. The offset PI curve and the cycle-through-a-peak figure are computed with this repository's own SCOOT-style model, which follows published descriptions; TRL's exact internal rules (for example congestion weighting and the 80% decrease threshold) are not published and may differ. Lab A1 and A2 numbers come from this repository's SUMO runs (seed 1) and illustrate the method, not any real SCOOT installation. Version dates follow published SCOOT histories.